

Project 4: Java Data Processor CI/CD Demo

Overview

This project focuses on building a standalone Java application (without Spring Boot) and managing heavy Maven dependencies. The key learning objective is **cached Maven `.m2` dependencies in Jenkins** to speed up subsequent pipeline runs.

Prerequisites

Ensure the following are installed and running on your Jenkins server:

- Docker
- Trivy
- Java 17 (JDK)
- Maven

Step 1: Network Setup

Create a shared Docker network for your infrastructure and deployments:

```
docker network create jenkins-net
```

Step 2: Infrastructure Setup

Run SonarQube and Jenkins on the `jenkins-net` network:

```
# Run SonarQube
```

```
docker run -d --name sonarqube -p 9000:9000 --network jenkins-net  
sonarqube:lts-community
```

```
# Run Jenkins (with Docker socket mounted)
```

```
docker run -d --name jenkins -p 8080:8080 -p 50000:50000 --network  
jenkins-net -v /var/run/docker.sock:/var/run/docker.sock -v  
jenkins_home:/var/jenkins_home jenkins/jenkins:lts
```

Step 3: Jenkins Configuration

1. Log into Jenkins.
2. Install necessary plugins: Docker Pipeline, SonarQube Scanner, Pipeline.
3. Configure SonarQube Server:
 - Go to Manage Jenkins -> System.
 - Add a SonarQube server named SonarQube and point it to `http://sonarqube:9000`.
4. Configure Global Tools:
 - Go to Manage Jenkins -> Tools.

- Add Maven and JDK installations if not relying on system defaults.

Step 4: Pipeline Execution

1. Create a new Pipeline job in Jenkins.
2. Choose "Pipeline script from SCM" or paste the contents of Jenkinsfile.
3. ****Run the build twice.**** Observe how the first run takes longer to download dependencies, while the second run is significantly faster because of the `-Dmaven.repo.local=${WORKSPACE}/.m2/repository`` flag we added to the Jenkinsfile!

Step 5: Verification

Check the logs of the deployed container to see the processor output:

- Command: ``docker logs java-data-processor-container``
- Expected Output: ``Data Processor finished successfully.``